微算機系統實習

第12組專案報告

LAB 10

組別： 12
班級、姓名與學號：
資工三 112590450 李馥亘
資工三 112590452 林希芸
資工三 112590454 林妍蓁
資工三 112590455 吳哲丞
日期：2025.12.22

1. 實驗內容：

- 透過 Python 程式即時讀取 ADC 感測器數值
- 使用 C++ 程式呼叫外部 Python 程式取得感測結果
- 整合 Lab9 的多執行緒核心運算功能，於條件成立時啟動運算流程
- 依據 ADC 感測結果決定是否執行核心運算與 LED 控制
- 使用 LED 顯示系統狀態（待命／運算中／完成）
- 練習感測資料同步、多執行緒控制與硬體輸出整合應用

2. 實驗過程及結果

實驗過程

- ADC 感測程式撰寫（get_adc.py）
    - 撰寫 Python 程式讀取 ADC（光敏電阻）數值
    - 每次執行程式即讀取一次 ADC，並將結果以 print() 輸出至標準輸出
    - 程式不進行邏輯判斷，僅負責取得即時感測值，方便被其他程式呼叫
- 主程式設計（main.cpp）
    - 使用 C++ 撰寫主程式，透過系統呼叫方式執行 Python 腳本
    - 每次呼叫 Python 程式時，取得最新的 ADC 感測數值並顯示於終端機
    - 搭配 Thread 機制，讓感測資料讀取不影響主程式的其他流程
    - 整合 Lab9 的多執行緒核心運算模組，當 ADC 感測數值符合設定條件時，啟動核心運算執行緒進行計算，並於運算完成後回傳結果
- 同步與測試流程
    - 重複執行主程式，確認每次呼叫皆能取得不同時間點的 ADC 值
    - 觀察光線變化時，ADC 數值能即時反映變動情形
    - 確認在運算期間感測數值持續更新，核心運算流程不影響系統即時性

預期實驗結果

- ADC 數值可即時更新並正確顯示
- 系統能依環境數值決定是否執行核心運算
- Lab9 的多執行緒運算功能成功整合至 Lab10
- LED 能清楚顯示待命、運算中與完成狀態
- 整體系統運作穩定，結果符合預期

實際上的結果

ADC 感測數值即時更新（運算前狀態）
系統運算進行中與 ADC 數值同步更新畫面
系統運算結束
ADC 感測數值同步更新與運算流程

遇到的問題&問題如何解決

- 網頁無法即時顯示數值
  - 傳統靜態網頁只會在首次載入時請求資料，因此後續無法得知伺服器內部變數（ADC、運算結果）的變化
  - 透過 AJAX 非同步更新機制，在網頁端加入 JavaScript 的 fetch 函式，每隔 80 毫秒自動向伺服器請求最新的 JSON 資料，達到局部資料即時更新的效果

3. 程式碼：

main.cpp

```
#include <iostream>
#include <stdio.h>
#include <stdlib.h>
#include <fstream>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <cstdio>
#include <arpa/inet.h>
#include <sys/socket.h>

using namespace std;

#define WEB_PORT 8080

// --- 全域變數與資源保護 ---
int g_adc_value = 0;
pthread_mutex_t count_mutex;
long long g_count = 0;
pthread_mutex_t g_mutex;

pthread_mutex_t led_mutex;
bool g_led = false;
```

```
const int THRESHOLD = 500;
const int TEST_LED = 398;

int g_working = 0;

// --- GPIO 工具函式 ---
bool file_exists(const string &path) {
   ifstream f(path);
   return f.good();
}

void gpio_export(int pin) {
   if (!file_exists("/sys/class/gpio/gpio" +
to_string(pin))) {
       ofstream exportFile("/sys/class/gpio/export");
       if (exportFile.is_open()) { exportFile << pin;
exportFile.close(); }
   }
}

void gpio_set_direction(int pin, const string &dir) {
   string path = "/sys/class/gpio/gpio" + to_string(pin) +
"/direction";
   ofstream dirFile(path);
   if (dirFile.is_open()) { dirFile << dir; dirFile.close();
}
}

void gpio_set_value(int pin, int value) {
   string path = "/sys/class/gpio/gpio" + to_string(pin) +
"/value";
   ofstream valFile(path);
   if (valFile.is_open()) { valFile << value;
valFile.close(); }
```

```
}

void calculation() {
   pthread_mutex_lock(&count_mutex);
   for (int i = 0; i < 2500000; i++) {
       g_count++;
   }
   pthread_mutex_unlock(&count_mutex);
}

// --- Thread A: 感測節點 ---
void* threadA_sensor(void* arg) {
   char buffer[128];
   while (true) {
       FILE* pipe = popen("python3 get_adc.py", "r");
       if (pipe) {
           if (fgets(buffer, sizeof(buffer), pipe) != NULL)
{
               int val = stoi(string(buffer));
               pthread_mutex_lock(&g_mutex);
               g_adc_value = val;
               pthread_mutex_unlock(&g_mutex);
           }
           pclose(pipe);
       }
       usleep(100000); // 0.1s
   }
   return NULL;
}

// --- Thread B: 運算與控制節點 ---
void* threadB_worker(void* arg) {
   gpio_export(TEST_LED);
   gpio_set_direction(TEST_LED, "out");
```

```
    while (true) {
        int current_adc;

        pthread_mutex_lock(&count_mutex);
        long long count = g_count;
        pthread_mutex_unlock(&count_mutex);
        if (count > 50000000) {
            pthread_mutex_lock(&g_mutex);
            g_working = 2;
            pthread_mutex_unlock(&g_mutex);
            continue;
        }

        pthread_mutex_lock(&g_mutex);
        current_adc = g_adc_value;
        pthread_mutex_unlock(&g_mutex);

        if (current_adc < THRESHOLD) {
            gpio_set_value(TEST_LED, 1);
            pthread_mutex_lock(&led_mutex); g_led = true;
pthread_mutex_unlock(&led_mutex);
            usleep(500000);
            pthread_mutex_lock(&g_mutex);
            if (g_working == 1) calculation();
            pthread_mutex_unlock(&g_mutex);
            gpio_set_value(TEST_LED, 0);
            pthread_mutex_lock(&led_mutex); g_led = false;
pthread_mutex_unlock(&led_mutex);
            usleep(500000);
            pthread_mutex_lock(&g_mutex);
            if (g_working == 1) calculation();
            pthread_mutex_unlock(&g_mutex);
        } else {
            gpio_set_value(TEST_LED, 1);
            pthread_mutex_lock(&led_mutex); g_led = true;
```

```
pthread_mutex_unlock(&led_mutex);
            usleep(2000000);
            gpio_set_value(TEST_LED, 0);
            pthread_mutex_lock(&led_mutex); g_led = false;
pthread_mutex_unlock(&led_mutex);
            usleep(2000000);
        }
    }
    return NULL;
}

// --- Thread C: Web Server ---
void* threadC_worker(void* arg) {
    int server_fd, client_fd;
    struct sockaddr_in addr;
    char buffer[4096];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) { perror("socket");
pthread_exit(NULL); }

    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(WEB_PORT);

    bind(server_fd, (struct sockaddr*)&addr, sizeof(addr));
    listen(server_fd, 5);

    printf("[Web] Server at http://localhost:%d\n",
WEB_PORT);

    while (1) {
        client_fd = accept(server_fd, NULL, NULL);
        if (client_fd < 0) continue;
```

```
        int adc, work, led;
        long long count;

        // 讀取共用變數
        pthread_mutex_lock(&g_mutex);
        adc = g_adc_value;
        work = g_working;
        pthread_mutex_unlock(&g_mutex);

        pthread_mutex_lock(&count_mutex);
        count = g_count;
        pthread_mutex_unlock(&count_mutex);

        pthread_mutex_lock(&led_mutex);
        led = g_led;
        pthread_mutex_unlock(&led_mutex);

        // 讀取請求
        memset(buffer, 0, sizeof(buffer));
        read(client_fd, buffer, sizeof(buffer)-1);

        // 處理按鈕切換
        if (strstr(buffer, "GET /?toggle=1")) {
            pthread_mutex_lock(&g_mutex);
            g_working = !g_working;
            pthread_mutex_unlock(&g_mutex);

            const char* redirect_response =
                "HTTP/1.1 302 Found\r\n"
                "Location: /\r\n"
                "Content-Length: 0\r\n"
                "\r\n";
            write(client_fd, redirect_response,
strlen(redirect_response));
            close(client_fd);
```

```
            continue;
        }

        // 處理 AJAX /status
        if (strstr(buffer, "GET /status")) {
            char json[256];
            sprintf(json,
                "{\"adc\": %d, \"led\": %d, \"work\": %d,
\"count\": %lld}",
                adc, led, work, count
            );

            char http_response[512];
            sprintf(http_response,
                "HTTP/1.1 200 OK\r\n"
                "Content-Type: application/json\r\n"
                "Content-Length: %lu\r\n"
                "\r\n"
                "%s",
                strlen(json), json
            );
            write(client_fd, http_response,
strlen(http_response));
            close(client_fd);
            continue;
        }

        // 產生 HTML
        char html[4096];
        sprintf(html,
            "<!DOCTYPE html>"
            "<html lang='zh-Hant'>"
            "<head>"
            "<meta charset='UTF-8'>"
            "<title>智慧環境運算監控系統</title>"
```

```
            "<style>"
                "body { font-family: Arial, sans-serif;
background-color: #f0f8ff; text-align: center; }"
                "h1 { color: #333; }"
                "p { font-size: 1.2em; }"
                ".status { font-weight: bold; color: %s; }"
                "button { padding: 10px 20px; font-size: 1em;
background-color: #4CAF50; color: white; border: none;
border-radius: 5px; cursor: pointer; }"
                "button:hover { background-color: #45a049; }"
                ".container { margin-top: 50px; }"
            "</style>"
            "<script>"
            "function updateStatus() {"
            "  fetch('/status').then(r => r.json()).then(data
=> {"
            "    document.getElementById('adc').innerText =
data.adc;"
            "    document.getElementById('led').innerText =
data.led ? 'ON' : 'OFF';"
            "    const statusMap = {0: '待機', 1: '運算中', 2:
'完成'};"
            "    document.getElementById('work').innerText =
statusMap[data.work] || '未知';"
            "    document.getElementById('count').innerText =
data.count;"
            "  });"
            "}"
            "setInterval(updateStatus, 80);"
            "</script>"
            "</head>"
            "<body>"
            "<div class='container'>"
                "<h1>智慧環境運算監控系統</h1>"
                "<p>亮度 ADC：<span id='adc'>%d</span></p>"
```

```
                "<p>LED 狀態：<span id='led'>%s</span></p>"
                "<p>運算狀態：<span id='work'>%s</span></p>"
                "<p>運算結果：<span id='count'>%lld</span></p>"
                "<form method='get'>"
                    "<button name='toggle' value='1'
type='submit'>切換狀態</button>"
                "</form>"
            "</div>"
            "</body>"
            "</html>",
            work ? "#008000" : "#ff0000",
            adc,
            led ? "ON" : "OFF",
            work == 2 ? "完成" : work == 1 ? "運算中" : "待機",
            count
        );

        char http_response[8192];
        sprintf(http_response,
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/html\r\n\r\n"
            "%s", html
        );

        write(client_fd, http_response,
strlen(http_response));
        close(client_fd);
    }
}

int main() {
    pthread_mutex_init(&g_mutex, NULL);
    pthread_mutex_init(&count_mutex, NULL);
    pthread_mutex_init(&led_mutex, NULL);
```

```
    pthread_t tidA, tidB, tidC;
    pthread_create(&tidA, NULL, threadA_sensor, NULL);
    pthread_create(&tidB, NULL, threadB_worker, NULL);
    pthread_create(&tidC, NULL, threadC_worker, NULL);

    printf("系統已啟動...\n");

    pthread_join(tidA, NULL);
    pthread_join(tidB, NULL);
    pthread_join(tidC, NULL);

    pthread_mutex_destroy(&g_mutex);
    pthread_mutex_destroy(&count_mutex);
    pthread_mutex_destroy(&led_mutex);

    return 0;
}
```

get_adc.py

```
import Jetson.GPIO as GPIO
import time

# =======================
# SPI 腳位設定 (BCM)
# =======================
SPICLK  = 11    # GPIO 11
SPIMISO = 9     # GPIO 9
SPIMOSI = 10    # GPIO 10
SPICS   = 8     # GPIO 8

# =======================
# GPIO 初始化
# =======================
```

```
GPIO.setmode(GPIO.BCM)
GPIO.setup(SPIMOSI, GPIO.OUT)
GPIO.setup(SPIMISO, GPIO.IN)
GPIO.setup(SPICLK, GPIO.OUT)
GPIO.setup(SPICS, GPIO.OUT)

# ======================
# MCP3008 ADC 讀取
# ======================
def read_adc(channel=0):
    if channel < 0 or channel > 7:
        return -1

    GPIO.output(SPICS, True)
    GPIO.output(SPICLK, False)
    GPIO.output(SPICS, False)

    command = channel
    command |= 0x18
    command <<= 3

    for _ in range(5):
        GPIO.output(SPIMOSI, bool(command & 0x80))
        command <<= 1
        GPIO.output(SPICLK, True)
        GPIO.output(SPICLK, False)

    result = 0
    for _ in range(12):
        GPIO.output(SPICLK, True)
        GPIO.output(SPICLK, False)
        result <<= 1
        if GPIO.input(SPIMISO):
            result |= 0x1
```

```python
    GPIO.output(SPICS, True)
    return result >> 1

# =======================
# Main：一次執行一次讀
# =======================
if __name__ == "__main__":
    try:
        adc_value = read_adc(0)
        print(adc_value)

    finally:
        GPIO.cleanup()
```

4. 本次實驗過程說明與解決方法：

本次實驗透過 Python 與 C++ 的整合實作 ADC 感測資料同步讀取，其中 Python 程式專注於即時擷取環境感測數值，C++ 主程式則負責呼叫感測程式、顯示結果，並整合 Lab9 的多執行緒核心運算功能。當 ADC 感測數值符合設定條件時，系統會啟動核心運算流程，並依據感測結果與運算狀態控制 LED 顯示系統待命、運算中與完成狀態。整體流程透過 Thread 機制避免阻塞主程式，使感測資料能持續更新且運算流程穩定執行，成功完成感測、運算與硬體控制的整合應用。

5. 分工：

學號、組員	貢獻比例	工作內容
112590450 李馥亘	33%	報告撰寫與整合
112590452 林希芸	0%	無
112590454 林妍蓁	33%	ADC 感測程式撰寫
112590455 吳哲丞	33%	程式測試與同步驗證